LAPORAN PRAKTIKUM PEMROGRAMAN WEB

MODUL 07: AJAX DAN KONSUMSI API

Identitas Praktikan:

- **Nama** : **Yensen Lawrenza Simangunsong**
- **NIM** : **103122430054**
- **Kelas** : **SE-08-02**
- **Program Studi** : **S1 Rekayasa Perangkat RPL**

I. Ringkasan Teknis Materi

1. Alur Kerja XMLHttpRequest (XHR)

- **Event Pemicu:** Tombol "Muat pengumuman" diklik oleh pengguna (event listener click).
- **Inisialisasi & Pengiriman:** Objek XMLHttpRequest dibuat menggunakan new XMLHttpRequest(). Permintaan disiapkan melalui xhr.open("GET", "pengumuman.txt", true) dengan mode asynchronous (non-blocking) dan dikirim menggunakan xhr.send().
- **Pemantauan Status & Response:** Event onreadystatechange memantau transisi state hingga readyState === 4 (DONE) dan memeriksa status HTTP 200. Jika berkas tidak ditemukan, kode mendeteksi status 404 dan menampilkan pesan kegagalan.
- **Pembaruan DOM:** Konten teks dari berkas dibaca melalui xhr.responseText dan dimasukkan ke elemen <pre> tanpa memuat ulang (reload) seluruh halaman web.

2. Alur Kerja Fetch API

- **Antarmuka Promise-Based:** Menggunakan fungsi fetch(url) yang mengembalikan Promise secara asinkron.
- **Pemeriksaan Status HTTP:** Karena Fetch tidak secara otomatis menolak (reject) status HTTP 4xx atau 5xx, dilakukan pemeriksaan eksplisit menggunakan if (!response.ok) throw new Error(...).
- **Parsing JSON:** Badan respons dibaca dan diubah menjadi data objek/array JavaScript menggunakan await response.json().
- **Pembaruan DOM:** Mengambil maksimal 3 peserta pertama menggunakan .slice(0, 3) dan menampilkannya sebagai daftar <li> secara dinamis ke dalam elemen <ul>, dilengkapi label nama sumber serta jumlah data yang berhasil ditampilkan.

3. Alur Kerja Axios (GET dan POST)

- **Axios GET:** Melakukan pengambilan data ke endpoint /api/users dengan konfigurasi batas waktu (timeout). Respons JSON otomatis terurai dan dapat diakses langsung melalui properti response.data.
- **Axios POST:** Mengirim data formulir registrasi pengguna baru dalam format JSON ({ name, email }) ke endpoint /api/users.
- **Pemisahan State:** Respons simulasi dari server (HTTP 201 Created, id 101, simulated: true) ditampilkan secara terpisah di bawah formulir dan tidak mengubah data pada daftar GET.

4. Peran State dan Lifecycle Effect pada React

- **useState:** Mengelola kondisi antarmuka secara reaktif, termasuk data pengguna (users), status loading, penanganan pesan error, nilai input formulir, dan hasil pengiriman POST.
- **useEffect:** Menjalankan pengambilan data otomatis saat komponen pertama kali dimuat (mount) dan mengeksekusi ulang setiap kali dependensi (skenario atau trigger reload) berubah.
- **Pembersihan (Cleanup) & AbortController:** Memanfaatkan fungsi cleanup pada useEffect dengan AbortController dan penanda aktif = false untuk membatalkan request lama saat pengguna mengganti skenario dengan cepat, sehingga mencegah terjadinya race condition dan memory leak.

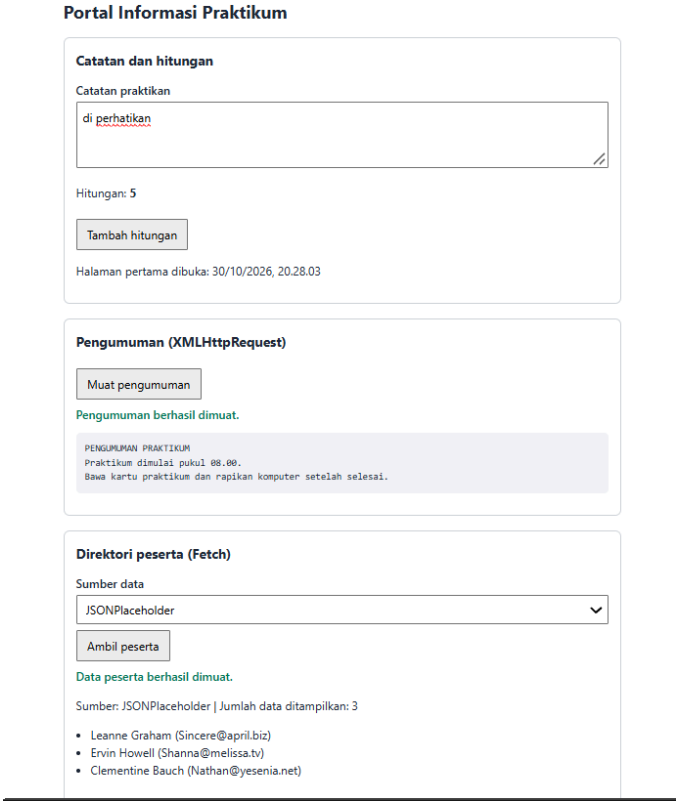
II. Tabel Hasil Pengujian Skenario

No	Modul Soal	Skenario Pengujian	Langkah / Input Pengujian	Hasil Aktual yang Teramati	Status
1	Soal 1	Pembaruan Isi Pengumuman (XHR)	Klik "Muat pengumuman", ubah teks pengumuman.txt, simpan, lalu klik lagi.	Teks pengumuman terbaru langsung tampil di elemen pre tanpa reload halaman.	Lolos
2	Soal 1	Penanganan Berkas Hilang (404)	Ubah sementara URL pengumuman menjadi hilang.txt pada app.js.	Muncul pesan error dengan status HTTP 404 pada elemen status.	Lolos
3	Soal 1	Pengujian Asinkron Non-Blocking	Klik "Ambil peserta" (Mock), lalu ketik di kolom catatan & klik "Tambah hitungan" 3x.	Nilai hitungan bertambah dari 0 ke 3, ketikan catatan tetap utuh, waktu buka awal tidak berubah, dan 3 peserta muncul setelah jeda 1.5 detik.	Lolos

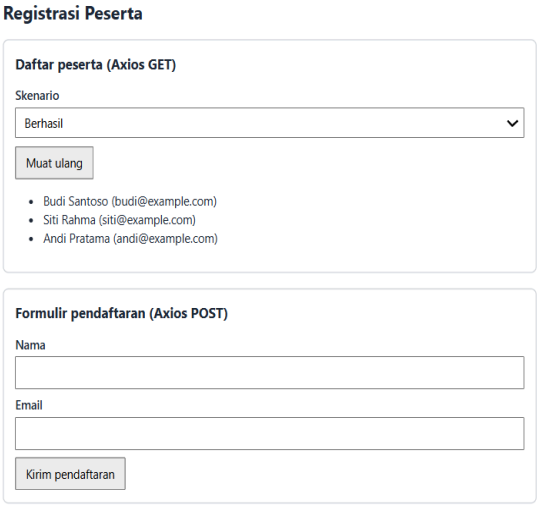
No	Modul Soal	Skenario Pengujian	Langkah / Input Pengujian	Hasil Aktual yang Teramati	Status
4	Soal 1	Pengambilan Data API Eksternal	Pilih sumber "JSONPlaceholder", lalu klik tombol "Ambil peserta".	Tepat 3 pengguna pertama tampil beserta email, nama sumber, dan jumlah data (3 data).	Lolos
5	Soal 2	Pemuatan GET Awal & Muat Ulang	Buka halaman Soal 02, lalu klik tombol "Muat ulang".	Data peserta termuat otomatis saat halaman dibuka via useEffect; tombol memicu request baru.	Lolos
6	Soal 2	Penanganan Data Kosong	Pilih opsi skenario "Data kosong" pada dropdown.	Menampilkan pesan "Data kosong: belum ada peserta", dan daftar lama dibersihkan.	Lolos
7	Soal 2	Penanganan Error HTTP 503	Pilih opsi skenario "Error HTTP" pada dropdown.	Menampilkan pesan error kegagalan server HTTP 503; daftar peserta dibersihkan.	Lolos
8	Soal 2	Penanganan Batas Waktu Timeout	Pilih opsi skenario "Timeout" (delay server 2500 ms, limit 700 ms).	Request dibatalkan tepat pada batas 700 ms dengan pesan error timeout; aplikasi dapat langsung digunakan kembali.	Lolos
9	Soal 2	Pembatalan Skenario Cepat (Race Condition)	Pilih skenario "Timeout", lalu segera pindah ke skenario "Berhasil".	Request timeout dibatalkan oleh AbortController; hasil akhir konsisten menampilkan data skenario "Berhasil".	Lolos
10	Soal 2	Validasi Input Formulir POST	Kirim formulir dengan nama kosong/spasi saja, atau format email tanpa karakter '@'.	Formulir menolak pengiriman dan memunculkan petunjuk perbaikan yang jelas.	Lolos
11	Soal 2	Pengiriman Registrasi POST Berhasil	Masukkan nama "Rina Praktikan" dan email "rina@example.com", lalu klik "Kirim pendaftaran".	Menampilkan detail respons: HTTP 201, ID 101, data sesuai input, simulated true, dan daftar GET tetap tidak terpengaruh.	Lolos

III. Dokumentasi Bukti Pengujian (Screenshots)

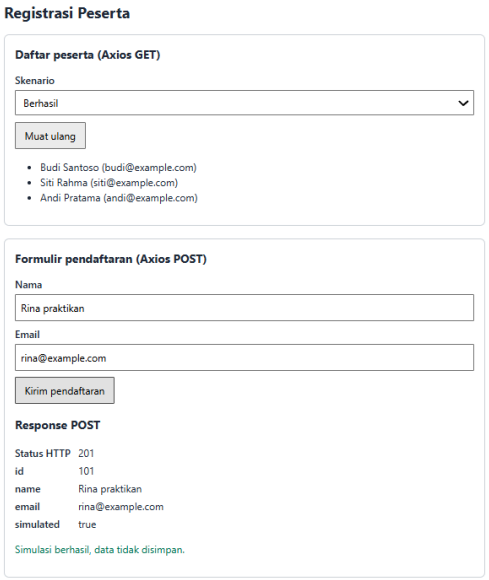
1. Tangkapan Layar Soal 01



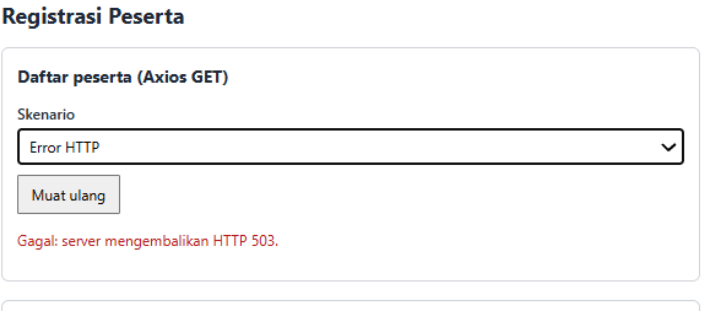
2. Tangkapan Layar Soal 02 (GET Berhasil)



3. Tangkapan Layar Soal 02 (Bukti Respons POST)



4. Tangkapan Layar Kondisi Gagal / Error



IV. Kesimpulan

Penerapan konsep AJAX memungkinkan aplikasi web memperbarui data secara asinkron tanpa memuat ulang seluruh halaman. Melalui XMLHttpRequest, Fetch API, dan Axios, pertukaran data dapat dilakukan secara efisien dengan manajemen status loading, penanganan kegagalan (error handling), serta pembatasan batas waktu (timeout). Pada React, pemanfaatan hook useState dan useEffect yang dilengkapi AbortController memastikan pembaruan tampilan antarmuka berjalan sinkron, stabil, dan bebas dari kendala race condition.